Team member:

24ECR183-SELVALAKSHITHA S

24ECR181-SARVIKA S

24ECR217-TAMILSELVAN R

Simulated Temperature Monitoring using Arduino, ESP32 and Raspberry Pi (MQTT)

Project Objective:

To simulate temperature monitoring using ESP32 as MQTT publisher and Raspberry Pi as MQTT broker and subscriber. Arduino can be used for serial monitoring demonstration.

Hardware Required:

1. Raspberry Pi (with SD card and internet connection)
2. ESP32
3. Arduino (optional for serial testing)
4. USB cables

Software Used:

1. Arduino (optional for serial testing)
2. Raspberry pi Imager(app)

Project Architecture:

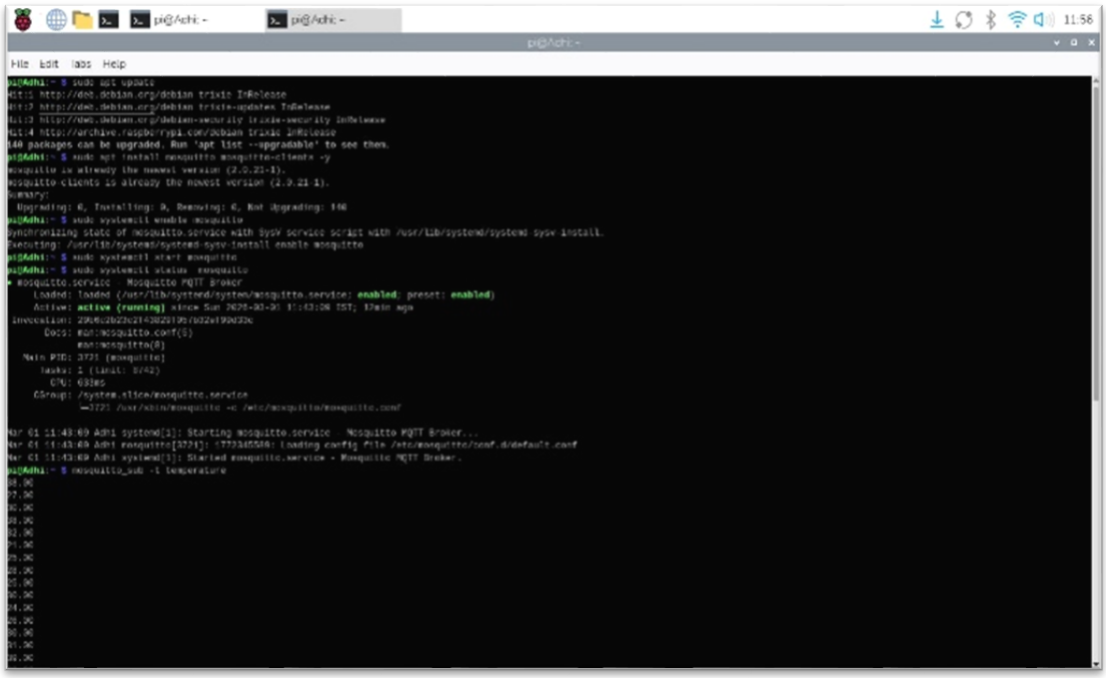
ESP32 → MQTT Publish → Raspberry Pi (Mosquitto Broker) → Subscriber (Python Script)

Procedure:

- *Setup of MQTT Broker on Raspberry Pi*

1. install Mosquito MQTT Broker on Raspberry Pi
   sudo apt install mosquitto mosquitto-clients -y
2. enable and start Mosquito MQTT Broker
   sudo systemctl enable mosquitto
   sudo systemctl start mosquito

3. Checking status of mosquito
   sudo systemctl status mosquito
   hostname -I
   Terminal-1: mosquitto_sub -t temperature
   Note: IP address – 192.168.83.79



4. Test MQTT locally by display the plain text on the terminal
   mosquitto_pub -t temperature -m "ece"

- ece is displayed in terminal 1
- Hence verified that MQTT started functioning locally

- *Install Required library in Ardunio IDE*

1. Open Ardunio IDE
2. Install ESP32 board package
3. Install PubSubClient library

- *ESP32 MQTT Publisher code:*

```
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "Surya's A33";
const char* password = "surya1234";
const char* mqtt_server = "10.68.183.52";
WiFiClient espClient;
PubSubClient client(espClient);
void setup() {
Serial.begin(115200);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
delay(1000);
Serial.println("Connecting to WiFi...");
}
Serial.println("Connected to WiFi");
client.setServer(mqtt_server, 1883);
}
void reconnect() {
while (!client.connected()) {
if (client.connect("ESP32Client")) {
Serial.println("Connected to MQTT");
} else {
delay(2000);
```

```
}
}
}
void loop() {
if (!client.connected()) {
reconnect();
}
client.loop();
float temperature = random(20, 40); // Simulated temperature
char tempString[8];
dtostrf(temperature, 1, 2, tempString);
client.publish("temperature", tempString);
Serial.println(tempString);
delay(5000);
}
```

- *Install Python MQTT Subscriber on Raspberry Pi*

1. Install pip and paho-mqtt- sudo apt install python3-pip -y
   pip3 install paho-mqtt

If environment error occur use:

```
pi@fsl02:~ $ pip3 install paho-mqtt
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.
```

```
sudo apt update

sudo apt install python3-venv python3-full -y

python3 -m venv mqttenv
source mqttenv/bin/activate
pip install paho-mqtt
```

2. Create subscriber file

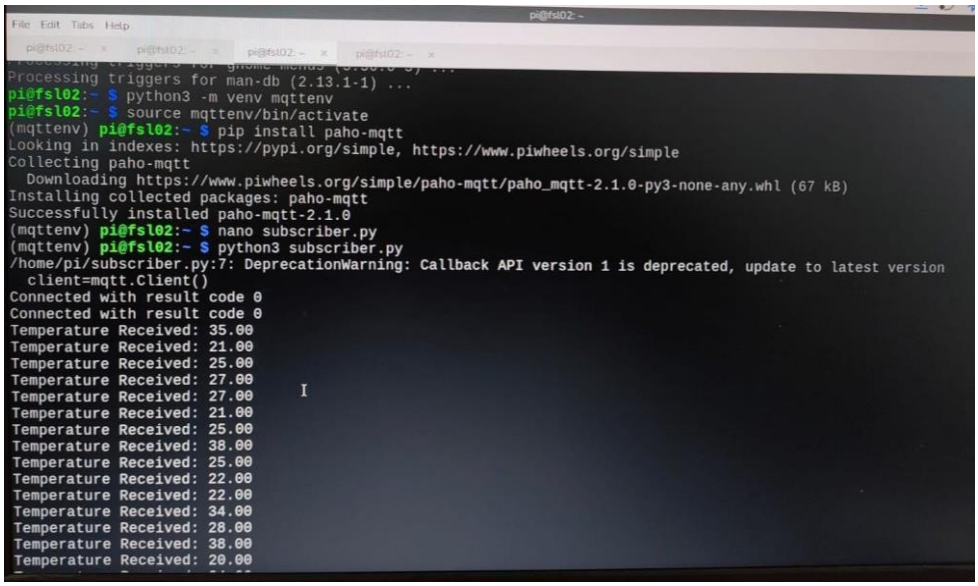
```
nano subscriber.py
import paho.mqtt.client as mqtt

def on_connect(client, userdata, flags, rc):
  print("Connected with result code", rc)
  client.subscribe("temperature")

def on_message(client, userdata, msg):
   print("Temperature Received:", msg.payload.decode())

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message
 client.connect("localhost", 1883, 60)
client.loop_forever()
```

3. Run subscriber file

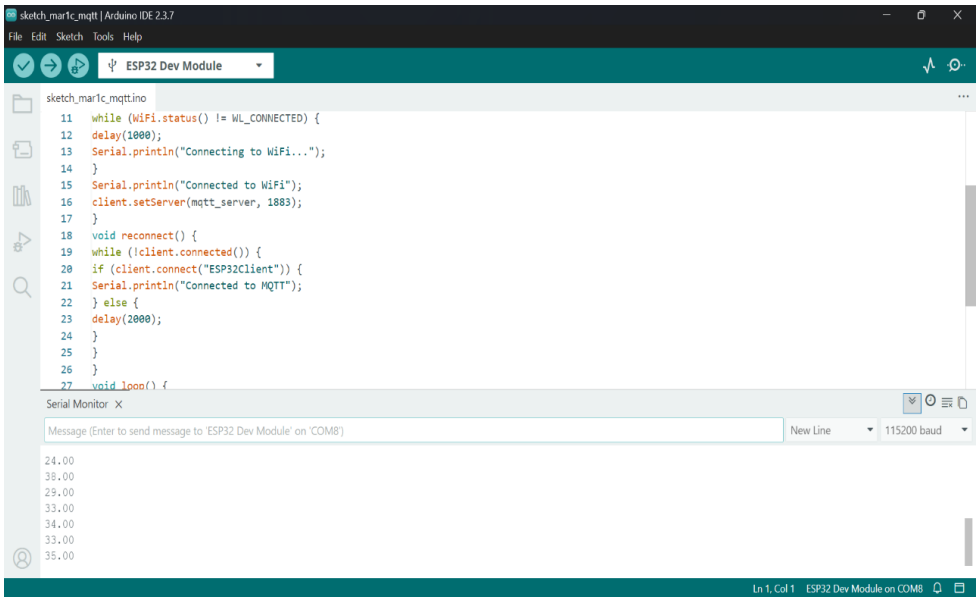
- *Working Explanation*

1. Raspberry Pi runs Mosquitto MQTT broker.
2. ESP32 connects to WiFi and publishes simulated temperature.
3. Raspberry Pi subscriber receives and displays temperature every 5 seconds.
4. System simulates IoT-based temperature monitoring.

Output:



```
Processing triggers for man-db (2.13.1-1) ...
pi@fsl02:~ $ python3 -m venv mqttenv
pi@fsl02:~ $ source mqttenv/bin/activate
(mqttenv) pi@fsl02:~ $ pip install paho-mqtt
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting paho-mqtt
  Downloading https://www.piwheels.org/simple/paho-mqtt/paho_mqtt-2.1.0-py3-none-any.whl (67 kB)
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-2.1.0
(mqttenv) pi@fsl02:~ $ nano subscriber.py
(mqttenv) pi@fsl02:~ $ python3 subscriber.py
/home/pi/subscriber.py:7: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client=mqtt.Client()
Connected with result code 0
Connected with result code 0
Temperature Received: 35.00
Temperature Received: 21.00
Temperature Received: 25.00
Temperature Received: 27.00
Temperature Received: 27.00
Temperature Received: 21.00
Temperature Received: 25.00
Temperature Received: 38.00
Temperature Received: 25.00
Temperature Received: 22.00
Temperature Received: 22.00
Temperature Received: 34.00
Temperature Received: 28.00
Temperature Received: 38.00
Temperature Received: 20.00
```



```
while (WiFi.status() != WL_CONNECTED) {
delay(1000);
Serial.println("Connecting to WiFi...");
}
Serial.println("Connected to WiFi");
client.setServer(mqtt_server, 1883);
}
void reconnect() {
while (!client.connected()) {
if (client.connect("ESP32Client")) {
Serial.println("Connected to MQTT");
} else {
delay(2000);
}
}
}
void loop() {
```

```
24.00
38.00
29.00
33.00
34.00
33.00
35.00
```

Result:

The temperature monitoring system using ESP32, Raspberry Pi, and MQTT was successfully implemented. The ESP32 published temperature data to the MQTT topic through Wi-Fi. The Raspberry Pi running the Mosquitto broker received the data, and the Python subscriber displayed the temperature values in real time. The output was also observed in the Arduino IDE serial monitor, confirming proper communication between all components